Product requirements document · revised v2 · 29 October 2026 · based on Prototype v1 (github.com/devashishb-rms/AutoJopps)

AutoJopps: Product Requirements

Automated New Job Opportunities and Smooth Job-Search Operations and Job Hops. A self-service SaaS agent that finds matching roles, tailors and submits applications on the user's behalf, pauses for the human only when needed, tracks outcomes, and learns from the user. This revision records what Prototype v1 implements, what is partial, and what is planned.

> **The name. AutoJopps** (pronounced auto-J-opps) is short for **Automated New Job Opportunities and Smooth Job-Search Operations and Job Hops**: receive frequent new job opportunities through automated, fast job applies. The product may also be called or named **AutoJobOpps**.

> **Reading guide.** Each requirement carries a status: Implemented exists and was exercised by the automated end-to-end tests in `samples/`; Partial exists in reduced form (noted); Planned is not in the code yet. Code references are file paths in the repository.

1. Vision and goals

Vision. Applying for jobs should take minutes of a person's attention a day, not hours, without giving up control of what goes out under their name.

Product goal (12 months). A paid, self-service product for Indian knowledge workers that applies to roles across major portals and company career sites within user-set caps, with a verified submit rate above 90% and a cost per application under ₹2.

Non-goals for v1. Recruiter-side features; interview preparation; resume writing as a standalone product; applying where a portal's terms forbid automation.

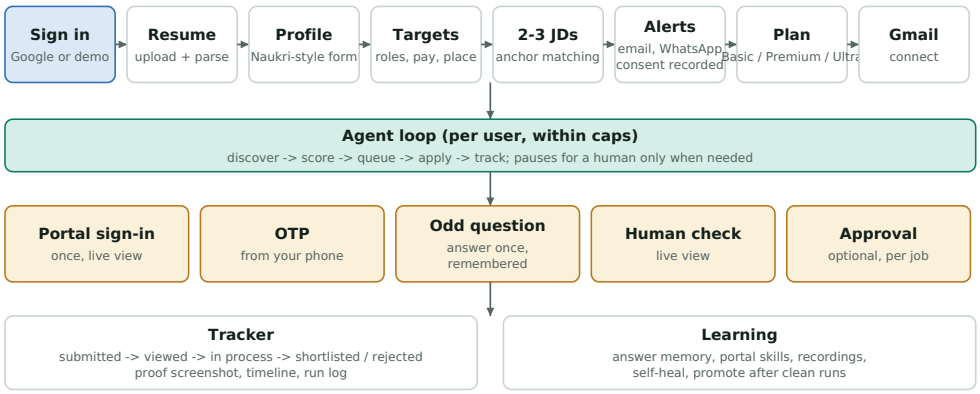
2. Users and personas

Persona	Situation	What they need	Plan fit
Active senior professional (8-20+ yrs)	Targeting 2-3 role types; long forms; cannot afford a wrong answer under their name	Precision, approval control, proof, few interruptions	Premium / Ultra
Mid-career engineer (3-8 yrs)	Applying broadly across portals while employed	Volume within caps, answers remembered, tracking	Basic / Premium
Returning or relocating candidate	Needs many applications quickly in a new city or after a break	Fast onboarding from a resume, WhatsApp alerts	Premium
Founder / operator (internal)	Runs the service	MRR, cost per application, HITL rate, skill health	Admin view

3. Problem

- Applying is repetitive: the same profile data and screening answers are typed into every portal and ATS.
- Existing auto-apply tools are opaque (users cannot see what was sent), break on portal changes, and ignore the steps that need a human (OTP, CAPTCHA, odd questions), so they silently fail.
- Senior candidates will not hand over their name to a bot without control and proof.
- India-specific: pricing in USD, no WhatsApp alerts, no Naukri-style profile model, no DPDP posture.

4. End-to-end user journey



The journey as implemented in Prototype v1: an 8-step setup, an agent loop with five human-help moments, tracking and learning.

5. Functional requirements

5.1 Account and onboarding

ID	Requirement	Status	Where / notes
FR-1	Sign up and sign in with Google (OIDC); same email as used on portals	Implemented	`auth.py`: Google start/callback; demo sign-in fallback when no client is configured
FR-2	Upload a resume (PDF, DOCX, TXT) and keep it for applications	Implemented	`profile.py`: stored under `data/uploads`; attached by the runner
FR-3	Parse the resume into a Naukri-style onboarding form with per-field confidence; user reviews every field	Implemented	`ai.py`: Claude when a key exists, heuristic parser otherwise; low-confidence fields flagged in the UI
FR-4	Review and set job types and skills	Implemented	Preferences step; skills drive matching
FR-5	Set designations, experience, compensation range, locations, work mode (remote / hybrid / WFO), relocation	Implemented	Preferences step; used as hard filters in `ai.match_score`
FR-6	Provide 2-3 target job descriptions	Implemented	Validated 1-3; keyword similarity anchors ranking and tailoring
FR-7	Give email and WhatsApp number for human-help alerts, with explicit permission	Implemented	Consent flags and timestamp stored (`Profile.notify`)
FR-8	Choose and pay for a plan: Basic ₹501 (5/day, 100/mo), Premium ₹1,001 (10/day, 200/mo), Ultra ₹2,001 (20/day, 400/mo)	Implemented	`billing.py`: Razorpay test-mode order + signature verification + webhook; demo activation when keys absent. Recurring subscriptions: Planned
FR-9	Sign in to the Gmail account used on portals	Partial	The human signs in on the live view; Google blocks automated sign-ins. Only the portal session is stored, never the Google password
FR-10	Authorise Gmail access so recruiter mails update tracking	Partial	Read-only scope behind a flag (Google Testing mode) with a recent-mail listing; the forwarding alias is a stub (no mail ingestion yet); Gmail-driven tracking updates and Google verification Planned

5.2 Matching and applying

ID	Requirement	Status	Where / notes
FR-11	Discover open roles on connected portals	Partial	`orchestrator.sync_jobs` pulls from the built-in JobHub portal; real portal adapters Planned
FR-12	Score every role against skills, target JDs, designation and hard filters; explain the score	Implemented	Reasons shown as tags; threshold 25; filtered roles kept as *skipped* with reason
FR-13	Tailor a headline and cover note per application	Implemented	Claude or template; shown on the application detail
FR-14	Apply automatically within the daily and monthly caps of the plan	Implemented	`billing.can_apply` checked before each queue; counted from *preparing* onward; one run per user at a time
FR-15	Answer screening questions from the profile; ask the user only when not confident	Implemented	Profile rules, answer memory, optional Claude draft; unsure -> *needs_human*
FR-16	Optionally require the user's approval before each submission	Partial	`User.approval_mode = approve_each`; the gate sits on the pre-submit check step, which JobHub always has. Adapters

			without that step need the gate moved before the final submit click
FR-17	Keep proof of every submission	Implemented	Full-page screenshot + portal reference id
FR-18	Run one application at a time per user; more runners on higher tiers	Partial	One runner thread per user today; tier runner counts are defined in `config.PLANS` but not yet scheduled

5.3 Human in the loop and remote access (requirement 12)

ID	Requirement	Status	Where / notes
FR-19	Pause for a human on portal login, OTP, CAPTCHA / human check, unknown question, approval; resume at the same step	Implemented	`runner.wait_for_human`; states *needs_human* / *awaiting_approval*
FR-20	Notify by in-app, email and WhatsApp (where consented) when help is needed	Implemented	`notify.py`; SMTP and WhatsApp Cloud API when configured, otherwise logged
FR-21	Let the user see and operate the bot's browser remotely (OTP, login, account creation, answers)	Implemented	Live view over WebSocket: screenshot stream + click / type / key relay; per-user authorisation
FR-22	Answer from the app without the live view where a value is enough (OTP, question, check word)	Implemented	HITL card; responses recorded in answer memory
FR-23	Time out and release a stuck run	Implemented	20-minute HITL timeout -> *failed*, retryable

5.4 Tracking and monitoring (requirement 11)

ID	Requirement	Status	Where / notes
FR-24	Show how many and which jobs were applied to, with a funnel	Implemented	Overview KPIs and funnel; Applications list with filters
FR-25	Show which applications the recruiter viewed, which are in process, and decisions	Partial	Polled from the portal status API (JobHub simulates it). Real portals: status pages / Gmail signals Planned
FR-26	Per-application timeline, run log, tailored content and proof	Implemented	Application detail panel
FR-27	Notification centre with delivery status per channel	Implemented	`/api/notifications`

5.5 Learning loop

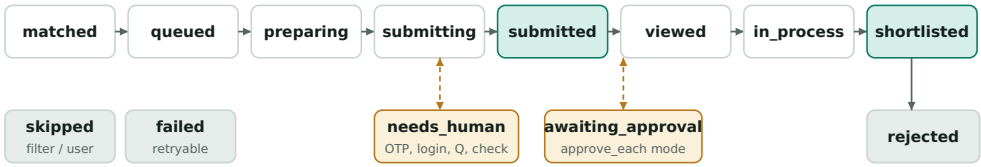
ID	Requirement	Status	Where / notes
FR-28	Remember confirmed answers and reuse them on later applications	Implemented	`AnswerMemory`; fuzzy match on normalised question; editable in Learning
FR-29	Record a user's demonstration in the live browser (with consent) and distil it into a draft portal skill	Implemented	Recorder in `runner.exec_cmd`, switched on by the user (record checkbox or demo sign-in); `/api/learning/recordings/{id}/distill`; passwords masked at capture. A separate consent record for recordings is Planned
FR-30	Versioned portal skills with status draft / trial / active / retired and success counters	Implemented	`Skill` table; promote and retire from the Learning page
FR-31	Self-heal when a portal changes: fall back to alternative locators, log the repair, file a trial skill version	Implemented	`runner.locate`; verified with the portal's "chaos" switch (8 repairs -> skill v2)
FR-32	Learn a brand-new portal from a teacher agent (Grok Bot or LLM agent) and compile it into a script	Planned	Adapter interface exists (skill steps); teacher integration not built
FR-33	Canary checks that replay skills daily on test accounts	Planned	

5.6 Operator (founder) view

ID	Requirement	Status	Where / notes
FR-34	MRR (gross and net), users, subscriptions by plan, applications by state, cost per application, HITL events, skill health	Implemented	`/api/admin/stats`; browser-minute and model-token cost meters
FR-35	Role-based access for the admin view	Planned	Today any signed-in user can open it (prototype)

6. Application lifecycle

Application state machine as implemented (db.Application.state). Cap check before queueing; counted from preparing onward.



States, with the two human states and their resume paths.

7. Non-functional requirements

Area	Requirement	v1 status
Security	Google password never stored; portal sessions encrypted per tenant; signed single-use live-view access; audit trail per application	Password never stored; sessions in a per-user vault file (plain in the prototype); live view authorised by session cookie and ownership; run log per application
Privacy (DPDP)	Notice and consent, purpose limitation, erasure, breach process, processor list	Consent with timestamp; delete-my-data removes resume, profile, answers, recordings, applications, run logs, proofs, portal sessions, subscriptions and payments; processors listed in the architecture document; breach runbook planned
Reliability	A run survives restarts or fails cleanly; no double submission	Runs in progress are failed on restart; cap counting prevents double counting; retry from failed
Performance	Live view frame every ~0.7 s; HITL response picked up within 1 s	Measured in tests
Scalability	Stateless control plane; runner fleet scales by users	Single process + SQLite in the prototype; target design in the architecture document
Cost	Cost per application tracked; target under ₹2 on scripted adapters	Metered per application (browser share + model tokens)
Accessibility and i18n	Keyboard-operable UI; English first, Hindi later	Keyboard works; English only

8. Plans and entitlements

Plan	Price / month (incl. 18% GST)	Daily cap	Monthly cap	Runners	Planner lane
Basic	₹501	5	100	1	no
Premium	₹1,001	10	200	2	yes
Ultra	₹2,001	20	400	4	yes

Caps are enforced in one place (`billing.can_apply`) on both windows. Net revenue after GST and about 2% gateway fee is about ₹4.16 per application at full cap on every tier.

9. Success metrics

Metric	Definition	Target (beta)
Verified submit rate	submitted with reference / started	≥ 90%
Human-help rate	HITL events per application	≤ 0.5 after the first week per portal
Time to first application	sign-up to first submitted	≤ 15 minutes
Cost per application	browser + model + messaging	≤ ₹2 (scripted adapters)
Activation	users who start the agent / sign-ups	≥ 60%
Paid conversion	paid / activated	≥ 25%
Reply rate	viewed or in process / submitted	tracked; benchmark in beta

10. Out of scope (v1) and constraints

- Portals whose terms prohibit automation (LinkedIn automation is off by default).
- Recruiter-side product, job posting, employer analytics.
- Automated Google sign-in (Google blocks it; the human signs in once).
- Consumer Claude or Grok subscriptions as the engine: model calls go through API keys under commercial terms.

11. Roadmap

Phase	Scope	Exit criteria
v1 (done)	Prototype on the mock portal: full journey, HITL, live view, caps, tracking, learning loop	Automated E2E passes; demoable
v1.1 (4-6 weeks)	Two real portal adapters (Naukri, one ATS family); Postgres; encrypted vault; Razorpay Subscriptions; admin auth; Gmail signals	25-user private beta; cost per application measured
v1.2 (8-12 weeks)	Cells and runner autoscaling; canary bot; WhatsApp templates approved; DPDP notice, retention and breach runbook; security review	Public launch; 90% verified submit rate
v2	Teacher agent (Grok Bot / LLM) for new portals, planner lane for Premium and Ultra, career-site adapters, Hindi UI	Cost per application ≤ ₹1.2; 5+ portals

12. Open questions

1. Which two real portals first (Naukri plus which ATS family)?
2. Do we keep GST-inclusive prices at ₹501 / 1,001 / 2,001?
3. Gmail: pursue Google verification (restricted scope, CASA) or stay on the forwarding alias?
4. Does Grok Bot expose an API or export its learned skills (affects FR-32 effort)?

Prepared 29 October 2026 from the repository state at commit bcbd67e. Statuses reflect the automated tests in `samples/api_smoke.py` and `samples/ui_e2e.py`.